

NUNNUN DB

0. 설계 기준

현재 개발 범위는 다음과 같다.

- 데모 로그인
- FCM 디바이스 등록
- 요일별 기상목표시간
- 기상목표 기반 취침 유도 알림
- 알림창 **잠자기** 버튼
- 수동 DND
- 깨우기 그룹
- 그룹 정원 4명
- 사용자당 깨우기 그룹 최대 1개
- 오늘의 포즈
- 깨우기
- 사진 인증
- AI 포즈 판정
- 최대 2회 인증
- **NEEDS_HELP**
- 성공 후 30분 쿨다운
- 성공 사진 8시간 유지
- **AWAKE / SLEEPING**
- 셀프 인증

이번 개발에서 제외:

- 여러 깨우기 그룹
- 그룹 탈퇴
- AI 친구

- 실제 결제
- 4명 → 8명 실제 확장
- 그룹 간 공유
- Roommate 신규 개발
- Reward

Roommate 기존 테이블은 삭제하지 않고 그대로 유지한다.

1. 최종 테이블 목록

번호	Domain	Table	처리
1	User	users	수정
2	Device	users_devices	유지
3	Auth	refresh_tokens	유지
4	Schedule	fixed_schedules	유지
5	Wake Target	weekly_wake_targets	신규
6	DND	dnd_windows	신규
7	Routine	daily_routines	일부 의미 변경
8	Sleep	sleep_sessions	수정
9	Sleep	sleep_feedbacks	유지
10	Wake	wake_groups	수정
11	Wake	wake_group_members	수정
12	Wake	daily_poses	신규
13	Wake	wake_requests	수정
14	Wake	wake_proofs	수정
15	Notification	notifications	수정
16	Roommate	roommate_groups	유지
17	Roommate	roommate_group_members	유지
18	Roommate	roommate_complaints	유지
19	Roommate	roommate_behavior_manuals	유지

총 19개 테이블.

기존 DB는 총 16개 테이블이었고 Wake 구조는 `wake_groups` → `wake_group_members` → `wake_requests` → `wake_proofs` 형태였다. DB(3).pdfPDF

2. USERS

기존 사용자 테이블을 최대한 유지한다. 기존에는 `id`, `nickname`, `email`, `password_hash`, `deleted_at` 을 저장하고 있었다. DB(3).pdfPDF

users

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	사용자 ID
<code>nickname</code>	VARCHAR(30)	NOT NULL	닉네임
<code>email</code>	VARCHAR(255)	UNIQUE, NOT NULL	이메일
<code>password_hash</code>	VARCHAR(255)	NOT NULL	비밀번호 해시
<code>avatar_url</code>	VARCHAR(512)	NULL	프로필 이미지
<code>is_demo</code>	BOOLEAN	NOT NULL, DEFAULT FALSE	데모 계정 여부
<code>deleted_at</code>	TIMESTAMP	NULL	회원탈퇴 시각

데모 계정

별도 `demo_accounts` 테이블을 만들지 않고:

```
users.is_demo = TRUE
```

인 사용자를 데모 계정으로 사용한다.

예:

```
id = 1
nickname = "눈눈"
email = "demo1@nunnun.local"
is_demo = true
```

`GET /demo-accounts` :

```
WHERE is_demo = TRUE
```

POST /auth/demo-login 의 demo_account_id 는 users.id 를 의미한다.

3. USERS_DEVICES

기존 구조 유지. DB(3).pdfPDF

users_devices

컬럼	타입	제약	설명
id	BIGINT	PK	디바이스 ID
user_id	BIGINT	FK, NOT NULL	사용자
fcm_token	VARCHAR(512)	UNIQUE, NOT NULL	FCM 토큰
platform	VARCHAR(20)	NOT NULL	ANDROID 등
created_at	TIMESTAMP	NOT NULL	등록 시각
updated_at	TIMESTAMP	NOT NULL	수정 시각

취침 유도 알림과 깨우기 알림을 FCM으로 보내기 위해 사용한다.

4. REFRESH_TOKENS

기존 구조 유지. DB(3).pdfPDF

refresh_tokens

컬럼	타입	제약	설명
id	BIGINT	PK	Token ID
user_id	BIGINT	FK, NOT NULL	사용자
token_hash	VARCHAR(255)	NOT NULL	Refresh Token Hash
expires_at	TIMESTAMP	NOT NULL	만료시각
revoked_at	TIMESTAMP	NULL	로그아웃/폐기
created_at	TIMESTAMP	NOT NULL	생성시각

데모 로그인도 동일한 JWT / Refresh Token 구조를 사용한다.

5. FIXED_SCHEDULES

fixed_schedules

컬럼	타입	제약	설명
id	BIGINT	PK	일정 ID
user_id	BIGINT	FK, NOT NULL	사용자
title	VARCHAR(100)	NOT NULL	일정명
day_of_week	VARCHAR(10)	NOT NULL	요일
start_time	TIME	NOT NULL	시작
end_time	TIME	NOT NULL	종료
created_at	TIMESTAMP	NOT NULL	생성
updated_at	TIMESTAMP	NOT NULL	수정

중요:

```
fixed_schedules
≠
DND
```

고정 시간표에서 DND를 자동 생성하지 않는다.

6. WEEKLY_WAKE_TARGETS

요일별 기상목표시간을 저장하는 신규 테이블.

weekly_wake_targets

컬럼	타입	제약	설명
id	BIGINT	PK	목표 ID
user_id	BIGINT	FK, NOT NULL	사용자
day_of_week	VARCHAR(10)	NOT NULL	요일
target_wake_time	TIME	NOT NULL	목표 기상시간
created_at	TIMESTAMP	NOT NULL	생성시각
updated_at	TIMESTAMP	NOT NULL	수정시각

핵심 제약

```
UNIQUE(user_id, day_of_week)
```

즉 한 사용자에게:

```
MONDAY → 하나  
TUESDAY → 하나  
...
```

만 존재한다.

예

```
user_id = 1  
day_of_week = MONDAY  
target_wake_time = 07:30
```

사용자가:

```
월요일, 06:30
```

으로 수정하면 새 row를 생성하지 않고:

```
07:30 → 06:30
```

으로 UPDATE한다.

앞으로 모든 월요일에 **06:30** 적용.

7. 기상목표 요일 제약

허용 값:

```
MONDAY  
TUESDAY  
WEDNESDAY  
THURSDAY
```

FRIDAY
SATURDAY
SUNDAY

가능하면:

```
CHECK (
    day_of_week IN (
        'MONDAY',
        'TUESDAY',
        'WEDNESDAY',
        'THURSDAY',
        'FRIDAY',
        'SATURDAY',
        'SUNDAY'
    )
)
```

를 적용한다.

8. DND_WINDOWS

사용자가 직접 입력한 방해금지 시간을 저장하는 신규 테이블.

dnd_windows

컬럼	타입	제약	설명
id	BIGINT	PK	DND ID
user_id	BIGINT	FK, NOT NULL	사용자
day_of_week	VARCHAR(10)	NOT NULL	요일
start_time	TIME	NOT NULL	시작시간
end_time	TIME	NOT NULL	종료시간
created_at	TIMESTAMP	NOT NULL	생성시각

예:

사용자 입력
월요일, 08:00~11:00

저장:

```
day_of_week = MONDAY
start_time = 08:00
end_time = 11:00
```

제약

```
CHECK(start_time < end_time)
```

```
UNIQUE(
    user_id,
    day_of_week,
    start_time,
    end_time
)
```

자정을 넘기는:

```
23:00~07:00
```

형태는 현재 허용하지 않는다.

DND는 상대방 깨우기만 차단한다.

다음은 차단하지 않는다.

```
셀프 인증
취침 유도 알림
```

9. DAILY_ROUTINES

기존 DB에는 날짜별 `target_bed_time`, `target_wake_time`, `estimated_return_time` 을 저장하고 있었다. DB(3).pdfPDF

요일별 기상목표의 실제 설정값은 이제:

```
weekly_wake_targets
```


가 기준이다.

하지만 `daily_routines.target_wake_time` 은 과거 통계용 스냅샷으로 남겨두는 것을 추천한다.

daily_routines

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	ID
<code>user_id</code>	BIGINT	FK, NOT NULL	사용자
<code>routine_date</code>	DATE	NOT NULL	날짜
<code>target_bed_time</code>	TIME	NULL	목표 취침시간
<code>target_wake_time</code>	TIME	NULL	해당 날짜에 적용된 기상목표 스냅샷
<code>estimated_return_time</code>	TIME	NULL	예상 귀가시간
<code>estimated_return_at</code>	TIMESTAMP	NULL	귀가 기준시각
<code>updated_at</code>	TIMESTAMP	NOT NULL	수정시각

UNIQUE

```
UNIQUE(user_id, routine_date)
```

중요한 차이

사용자가 직접 매일 `daily_routines.target_wake_time` 을 설정하지 않는다.

기준:

```
weekly_wake_targets
```

에서 해당 날짜의 목표를 가져와 필요할 때:

```
daily_routines.target_wake_time
```

에 당시 값을 스냅샷으로 저장한다.

이유:

월요일 07:30으로 기상
↓
나중에 월요일 설정을 09:00으로 변경

하더라도 과거 월요일의 통계를 07:30 기준으로 계산할 수 있어야 하기 때문이다.

10. SLEEP_SESSIONS

기존 DB에서는 잠들기 버튼을 누른 시각을 `started_at` 으로 기록하고 있었다.
DB(3).pdfPDF

이번에는 알림창에서 잠자기를 누르는 경우를 구분할 수 있도록 `source` 를 추가한다.

sleep_sessions

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	수면 ID
<code>user_id</code>	BIGINT	FK, NOT NULL	사용자
<code>sleep_date</code>	DATE	NOT NULL	수면 기준 날짜
<code>started_at</code>	TIMESTAMP	NOT NULL	잠자기 누른 시간
<code>source</code>	VARCHAR(20)	NOT NULL, DEFAULT APP	잠자기 진입 경로
<code>created_at</code>	TIMESTAMP	NOT NULL	생성시각

source

APP
NOTIFICATION

예:

앱 내부에서 누름:

`source = APP`

휴대폰 알림창의 [잠자기] 클릭:

```
source = NOTIFICATION
```

11. SLEEP_FEEDBACKS

기존 구조 그대로 유지. DB(3).pdfPDF

sleep_feedbacks

컬럼	타입	제약	설명
id	BIGINT	PK	ID
user_id	BIGINT	FK, NOT NULL	사용자
feedback_date	DATE	NOT NULL	날짜
score	VARCHAR(20)	NOT NULL	만족도
created_at	TIMESTAMP	NOT NULL	생성일

Score:

```
VERY_BAD  
BAD  
NORMAL  
GOOD  
VERY_GOOD
```

제약:

```
UNIQUE(user_id, feedback_date)
```

12. WAKE_GROUPS

기존 테이블에는 그룹 ID, 이름, 초대코드, 생성자 등이 있었다. DB(3).pdfPDF

capacity 를 추가한다.

wake_groups

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	그룹 ID
<code>name</code>	VARCHAR(50)	NOT NULL	그룹명
<code>invite_code</code>	VARCHAR(6)	UNIQUE, NOT NULL	초대코드
<code>creator_id</code>	BIGINT	FK, NOT NULL	생성자
<code>capacity</code>	SMALLINT	NOT NULL, DEFAULT 4	정원
<code>created_at</code>	TIMESTAMP	NOT NULL	생성일

현재:

```
capacity = 4
```

향후 확장성을 위해:

```
CHECK(capacity IN (4, 8))
```

를 둘 수 있다.

현재는 실제 **8명 확장** 기능을 구현하지 않는다.

13. WAKE_GROUP_MEMBERS

기존 구조는 슬롯 **1~12**였으며 한 사용자가 여러 그룹에 가입할 수 있었다. DB(3).pdfPDF
현재는 사용자당 깨우기 그룹 최대 1개.

wake_group_members

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	멤버십 ID
<code>wake_group_id</code>	BIGINT	FK, NOT NULL	그룹
<code>user_id</code>	BIGINT	FK, NOT NULL	사용자
<code>slot_no</code>	SMALLINT	NOT NULL	그룹 슬롯
<code>joined_at</code>	TIMESTAMP	NOT NULL	참여시각

제약

```
UNIQUE(user_id)
```

사용자당 깨우기 그룹 최대 1개.

```
UNIQUE(wake_group_id, user_id)
```

같은 그룹 중복 참여 방지.

```
UNIQUE(wake_group_id, slot_no)
```

슬롯 중복 방지.

```
CHECK(slot_no BETWEEN 1 AND 8)
```

향후 8명까지 지원 가능한 구조.

현재는:

```
wake_groups.capacity = 4
```

이므로 Service에서 1~4 슬롯만 배정한다.

14. DAILY_POSES

그룹별 오늘의 포즈를 저장한다.

daily_poses

오늘의 포즈 생성 방식

- 미리 준비한 여러 개의 포즈 이미지가 있음
- 하루에 그룹별로 그중 하나를 랜덤 선택
- 선택된 포즈를 그날의 기준 포즈로 사용
- 사용자가 인증 사진을 찍으면 선택된 기준 포즈 이미지와 비교
- AI/Vision 모델은 "두 이미지의 포즈가 얼마나 유사한지" 점수만 계산

- 70점 이상 SUCCESS, 미만 FAIL

즉 기존 `daily_poses.pose_description` 중심 구조보다 **이미지 기준 포즈를 저장하는 구조**로 바꾸는 게 더 맞아.

DB도 이렇게 수정하는 걸 추천해

`daily_poses` :

컬럼	타입	설명
<code>id</code>	BIGINT	PK
<code>wake_group_id</code>	BIGINT	그룹
<code>pose_date</code>	DATE	날짜
<code>pose_key</code>	VARCHAR(50)	포즈 식별자
<code>pose_image_url</code> 또는 <code>pose_image_object_key</code>	VARCHAR(512)	기준 포즈 이미지
<code>created_at</code>	TIMESTAMP	생성시각

제약:

```
UNIQUE(wake_group_id, pose_date)
```

예:

```
pose_key = "POSE_03"
pose_image_object_key = "poses/pose_03.png"
```

그리고 포즈 원본이 완전히 고정된 세트라면 더 깔끔하게 **별도** `poses` **마스터 테이블**을 둘 수도 있어.

```
poses
- id
- code
- image_object_key
- description
- active
```

그럼 `daily_poses` 는:

```
daily_poses
- id
```

- wake_group_id
- pose_date
- pose_id
- created_at

만 가지면 돼.

이쪽이 더 좋아. 왜냐하면 같은 이미지 URL/경로를 매일 복사해서 저장할 필요가 없거든.

15. WAKE_REQUESTS

기존 DB에서는 `SENT / VERIFIED / EXPIRED` 상태였다. DB(3).pdfPDF

현재는 최대 2회 인증과 `NEEDS_HELP` 를 반영한다.

wake_requests

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	요청 ID
<code>wake_group_id</code>	BIGINT	FK, NOT NULL	그룹
<code>sender_id</code>	BIGINT	FK, NOT NULL	깨운 사람
<code>receiver_id</code>	BIGINT	FK, NOT NULL	깨워지는 사람
<code>status</code>	VARCHAR(20)	NOT NULL	요청 상태
<code>attempt_count</code>	SMALLINT	NOT NULL, DEFAULT 0	인증 제출 횟수
<code>requested_at</code>	TIMESTAMP	NOT NULL	요청 시각
<code>created_at</code>	TIMESTAMP	NOT NULL	생성일

status

SENT
VERIFIED
NEEDS_HELP

현재 플로우에서는 `EXPIRED` 를 사용하지 않는다.

attempt_count

```
0 = 사진 제출 전  
1 = 1회차 제출 완료  
2 = 2회차 제출 완료
```

```
CHECK(attempt_count BETWEEN 0 AND 2)
```

남은 인증 횟수:

```
remaining_attempts  
= 2 - attempt_count
```

DB에 따로 저장하지 않는다.

16. 셀프 인증

별도 테이블을 만들지 않는다.

셀프 인증:

```
sender_id = receiver_id
```

예:

```
sender_id = 1  
receiver_id = 1
```

일반 `wake_requests` 와 `wake_proofs` 흐름을 그대로 재사용한다.

17. WAKE_PROOFS

기존에는 성공 인증만 고려해 이미지, 인증시각, 만료시각이 모두 NOT NULL이었다.

DB(3).pdfPDF

실패 판정도 저장해야 하므로 수정한다.

`wake_proofs`

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	인증 ID
<code>wake_request_id</code>	BIGINT	UNIQUE, FK, NOT NULL	WakeRequest
<code>image_object_key</code>	VARCHAR(512)	NULL	성공 사진 S3 Key
<code>pose_match_score</code>	SMALLINT	NOT NULL	AI 점수
<code>pose_match_result</code>	VARCHAR(20)	NOT NULL	SUCCESS / FAIL
<code>submitted_at</code>	TIMESTAMP	NOT NULL	최근 제출시각
<code>verified_at</code>	TIMESTAMP	NULL	성공시각
<code>expires_at</code>	TIMESTAMP	NULL	사진 만료시각
<code>created_at</code>	TIMESTAMP	NOT NULL	최초 생성
<code>updated_at</code>	TIMESTAMP	NOT NULL	최근 수정

제약

```
UNIQUE(wake_request_id)
```

```
CHECK(pose_match_score BETWEEN 0 AND 100)
```

```
pose_match_result:
SUCCESS
FAIL
```

AI 성공 기준:

```
score >= 70 → SUCCESS
score < 70 → FAIL
```

70 자체는 DB에 저장하지 않고 서버 설정값으로 관리한다.

18. 인증 재시도 저장 방식

WakeRequest 하나당 Proof 하나를 유지한다.

1회차

```
wake_proofs INSERT  
wake_requests.attempt_count = 1
```

첫 번째 FAIL

```
status = SENT  
pose_match_result = FAIL
```

재시도 가능.

2회차

기존 Proof:

```
UPDATE
```

한다.

```
wake_requests.attempt_count = 2
```

2회차도 실패:

```
wake_requests.status = NEEDS_HELP
```

19. SUCCESS / 쿨다운 / 사진

인증 성공:

```
wake_requests.status = VERIFIED
```

```
wake_proofs.pose_match_result = SUCCESS  
wake_proofs.verified_at = 실제 성공시각
```

```
wake_proofs.expires_at = verified_at + 8시간
```

30분 쿨다운은 별도 컬럼을 만들지 않는다.

기존 DB에서도 `verified_at + 30분` 으로 계산하도록 설계되어 있었다. DB(3).pdfPDF

```
cooldown_until  
= verified_at + 30분
```

20. 인증사진 삭제

성공 사진:

```
verified_at + 8시간
```

후 S3에서 삭제한다.

하지만 이제는 **WakeProof row** 자체는 삭제하지 않는다.

사진만:

```
image_object_key = NULL
```

로 변경한다.

유지:

```
pose_match_score  
pose_match_result  
submitted_at  
verified_at  
expires_at
```

이 데이터는 기상 기록 및 통계에 사용할 수 있다.

21. NOTIFICATIONS

기존 DB에서 가장 크게 보완되는 부분이다.

원래 `notifications` 에는 이미 `user_id` , `type` , `title` , `body` , `reference_id` , `scheduled_at` , `sent_at` , `status` 가 있었고 `PENDING / SENT / FAILED / CANCELLED` 상태도 정의되어 있었다.
DB(3).pdfPDF

여기에 어느 기상목표를 위한 취침 알람인지 구분할 `target_wake_at` 을 추가한다.

notifications

컬럼	타입	제약	설명
<code>id</code>	BIGINT	PK	알림 ID
<code>user_id</code>	BIGINT	FK, NOT NULL	수신 사용자
<code>type</code>	VARCHAR(50)	NOT NULL	알림 종류
<code>title</code>	VARCHAR(100)	NOT NULL	제목
<code>body</code>	VARCHAR(500)	NULL	알림 내용
<code>reference_id</code>	BIGINT	NULL	관련 객체 ID
<code>target_wake_at</code>	TIMESTAMP	NULL	이 알림이 속한 기상목표 시각
<code>scheduled_at</code>	TIMESTAMP	NULL	예약 발송시각
<code>sent_at</code>	TIMESTAMP	NULL	실제 발송시각
<code>status</code>	VARCHAR(20)	NOT NULL	알림 상태
<code>created_at</code>	TIMESTAMP	NOT NULL	생성시각

22. Notification Type

현재 핵심 Type:

BEDTIME_REMINDER
WAKE_REQUEST

BEDTIME_REMINDER

취침 유도 알람.

특징:

무음
무진동
[잠자기] 버튼

무음/무진동 자체는 Android의 Notification Channel에서 처리한다.

DB에서는:

```
type = BEDTIME_REMINDER
```

인지만 구분하면 된다.

WAKE_REQUEST

다른 사용자가 보낸 깨우기 알림.

취침 유도 알림과 별도 Notification Channel을 사용한다.

23. Notification Status

PENDING
SENT
FAILED
CANCELLED

PENDING

아직 발송되지 않은 예약 알림.

SENT

FCM 발송 완료.

FAILED

FCM 발송 실패.

CANCELLED

발송 예정이었으나 더 이상 보내면 안 되는 알림.

예:

잠자기 버튼 클릭
기상목표 변경
기상목표 삭제

24. TARGET_WAKE_AT

`target_wake_at` 은 취침 유도 알림의 사이클 ID 역할을 한다.

예:

```
target_wake_at  
= 2026-08-17 09:00
```

해당 기상목표의 알림:

```
00:00  
01:30  
03:00  
04:30  
06:00  
07:30
```

모두 같은:

```
target_wake_at = 2026-08-17 09:00
```

을 가진다.

따라서 사용자가 `03:00` 알림에서 잠자기를 누르면:

```
WHERE user_id = ?  
  AND type = 'BEDTIME_REMINDER'  
  AND target_wake_at = ?  
  AND status = 'PENDING'
```

인 알림들을:

```
CANCELLED
```

로 바꿀 수 있다.

25. 취침 유도 알림 생성

기상목표를 `T` 라고 한다.

첫 알림
= T - 9시간

이후:

90분 간격

마지막:

T - 1시간 30분

총 최대 6개.

예:

T = 09:00

생성:

scheduled_at	body
00:00	취침 1시간 전이에요.
01:30	기상까지 7시간 30분 남았어요.
03:00	기상까지 6시간 남았어요.
04:30	기상까지 4시간 30분 남았어요.
06:00	기상까지 3시간 남았어요.
07:30	기상까지 1시간 30분 남았어요.

모든 row:

```
type = BEDTIME_REMINDER
target_wake_at = 해당 목표
status = PENDING
```

26. 알림 중복 방지

동일한 알림이 여러 번 생성되지 않도록:

```
UNIQUE(  
    user_id,  
    type,  
    target_wake_at,  
    scheduled_at  
)
```

을 권장한다.

즉 같은 사용자에게:

```
2026-08-17 09:00 기상  
+  
03:00 취침 알림
```

이 두 번 생성될 수 없다.

27. 잠자기 버튼 처리

알림의:

[잠자기]

클릭:

```
POST /me/sleep
```

서버 처리:

```
1. sleep_sessions INSERT  
2. started_at = 현재시간  
3. source = NOTIFICATION  
4. 현재 target_wake_at에 해당하는  
   남은 PENDING BEDTIME_REMINDER 조회  
5. status = CANCELLED
```

이미 발송된:

SENT

알림은 그대로 둔다.

다음 기상목표를 위한 알림도 건드리지 않는다.

28. 다음날 알림

예:

```
오늘 target_wake_at  
= 2026-08-17 09:00
```

잠자기 클릭으로 오늘 남은 알림을 취소했다고 해도:

```
다음 target_wake_at  
= 2026-08-18 09:00
```

알림들은 별도 row이므로 그대로 유지한다.

즉:

```
오늘 사이클 CANCELLED  
≠  
알림 기능 OFF
```

이다.

29. 기상목표 변경 시

예:

```
기존  
화요일 09:00  
  
변경  
화요일 08:00
```

변경되기 전 목표를 위한 아직 발송되지 않은 알림:

```
status = PENDING
```

을:

```
status = CANCELLED
```

로 변경한다.

그리고 새로운:

```
target_wake_at = 화요일 08:00
```

기준으로 다시 알림을 생성한다.

단 이미 지난 예약시간은 생성하지 않는다.

30. 기상목표 삭제 시

요일별 목표를 삭제하면:

```
weekly_wake_targets row DELETE
```

그리고 삭제된 목표에 연결된 아직 발송 전인:

```
BEDTIME_REMINDER  
+  
PENDING
```

알림은:

```
CANCELLED
```

처리한다.

31. NEEDS_HELP

DB에 직접 저장되는 경우:

```
인증 2회 모두 실패
```

```
wake_requests.status = NEEDS_HELP
```

반면:

```
기상목표 +15분까지 성공 인증 없음
```

은 별도 WakeRequest가 없을 수도 있으므로 그룹 상세 조회 시 계산할 수 있다.

조건:

```
현재 >= 오늘 목표 + 15분  
AND  
오늘 SUCCESS 없음
```

이면:

```
state = NEEDS_HELP
```

기상목표가 없는 요일에는 적용하지 않는다.

32. AWAKE / SLEEPING

DB에 상태 컬럼을 만들지 않는다.

조회 시 계산.

성공 인증:

```
AWAKE
```

다음 기상목표의:

8시간 전

부터:

SLEEPING

예:

다음 목표 = 09:00

01:00부터 SLEEPING

취침 알림과 연결하면:

00:00 → 첫 취침 알림
01:00 → SLEEPING 기준시각
01:30 → 두 번째 취침 알림

이다.

33. 그룹 카드에서 DB에 저장하지 않는 값

다음 값은 저장하지 않고 조회 시 계산한다.

```
remaining_to_target  
next_target_at  
actual_wake_time 표시 문자열  
AWAKE  
SLEEPING  
can_wake  
block_reason  
wake_available_at  
cooldown_until  
remaining_attempts
```

예:

actual_wake_time

```
= wake_proofs.verified_at
```

```
cooldown_until  
= verified_at + 30분
```

```
remaining_attempts  
= 2 - attempt_count
```

34. NOTIFICATIONS와 기존 `reference_id`

기존 `reference_id` 는 그대로 유지한다.

깨우기 알림:

```
type = WAKE_REQUEST  
reference_id = wake_requests.id
```

취침 유도 알림은 굳이 다른 객체를 참조하지 않아도 된다.

```
type = BEDTIME_REMINDER  
reference_id = NULL  
target_wake_at = 2026-08-17 09:00
```

처럼 사용할 수 있다.

35. Roommate

기존:

```
roommate_groups  
roommate_group_members  
roommate_complaints  
roommate_behavior_manuals
```

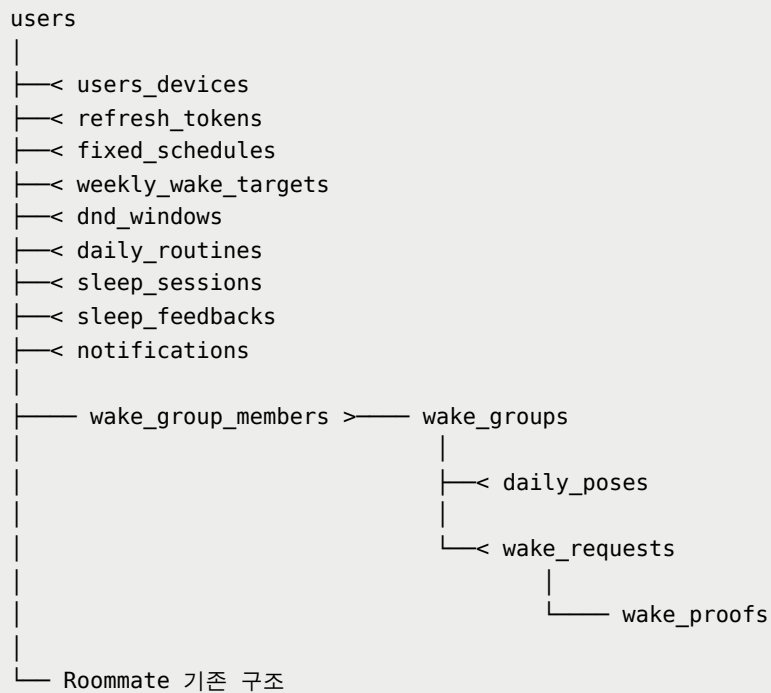
는 수정하지 않는다.

기존 DB는 룸메이트 멤버 최대 2명과 불만사항/행동 매뉴얼 구조를 이미 갖고 있다.

DB(3).pdfPDF DB(3).pdfPDF

현재 개발에서는 사용하지 않을 뿐이다.

36. 핵심 FK 관계



Wake:

```
users.id
├─ wake_requests.sender_id
└─ wake_requests.receiver_id
```

```
wake_groups.id
├─ wake_group_members.wake_group_id
├─ daily_poses.wake_group_id
└─ wake_requests.wake_group_id
```

```
wake_requests.id
└─ wake_proofs.wake_request_id
```

기존 DB에서도 `users`, `wake_groups`, `wake_requests`, `wake_proofs` 가 이러한 핵심 FK 관계로 연결돼 있었다. DB(3).pdfPDF DB(3).pdfPDF

37. 핵심 UNIQUE / CHECK 모음

```
-- 요일별 기상목표 하나
UNIQUE(user_id, day_of_week)

-- 동일 DND 중복 방지
UNIQUE(user_id, day_of_week, start_time, end_time)

CHECK(start_time < end_time)
```

```
-- 사용자당 Wake Group 하나
UNIQUE(user_id)
```

```
-- 그룹 구성원 중복 방지
UNIQUE(wake_group_id, user_id)
```

```
-- 그룹 슬롯 중복 방지
UNIQUE(wake_group_id, slot_no)

CHECK(slot_no BETWEEN 1 AND 8)
```

```
-- 그룹별 하루 포즈 하나
UNIQUE(wake_group_id, pose_date)
```

```
-- 인증 최대 2회
CHECK(attempt_count BETWEEN 0 AND 2)
```

```
-- 요청당 Proof 하나
UNIQUE(wake_request_id)
```

```
-- AI 점수
CHECK(pose_match_score BETWEEN 0 AND 100)
```

```
-- 취침 알림 중복 생성 방지
UNIQUE(
    user_id,
    type,
    target_wake_at,
```

```

    scheduled_at
)

```

38. 기존 DB에서 변경할 부분

Table	변경
users	avatar_url , is_demo 추가
users_devices	변경 없음
refresh_tokens	변경 없음
fixed_schedules	변경 없음
weekly_wake_targets	신규
dnd_windows	신규
daily_routines	target_wake_time 을 날짜별 설정값이 아닌 스냅샷으로 사용
sleep_sessions	source 추가
sleep_feedbacks	변경 없음
wake_groups	capacity 추가
wake_group_members	UNIQUE(user_id) , slot 1~8
daily_poses	신규
wake_requests	attempt_count , NEEDS_HELP
wake_proofs	AI 점수/결과/재시도 구조 추가
notifications	target_wake_at 추가 + 취침 알림 정책 적용
roommate_*	변경 없음

39. 취침 알림 전체 DB 흐름

```

사용자가
화요일, 09:00
등록
↓
weekly_wake_targets 저장
↓
다음 화요일 09:00
target_wake_at 계산
↓
notifications 생성

```



```
00:00 PENDING
01:30 PENDING
03:00 PENDING
04:30 PENDING
06:00 PENDING
07:30 PENDING
```

↓

예약시간 도착

↓

FCM 전송

↓

SENT

사용자가 03:00 알람에서:

[잠자기]

클릭:

```
sleep_sessions INSERT
```

```
started_at = 03:00
```

```
source = NOTIFICATION
```

그리고:

```
04:30 CANCELLED
```

```
06:00 CANCELLED
```

```
07:30 CANCELLED
```

다음 기상목표의 알람은 그대로 유지.

40. 현재 기준 핵심 정리

이번 DB에서 새로 추가되는 핵심 테이블은:

```
weekly_wake_targets
dnd_windows
daily_poses
```

기존 테이블 중 크게 변경되는 것은:

```
wake_requests  
wake_proofs  
notifications
```

이고, 특히 이번 알림 정책 때문에 `notifications.target_wake_at` 이 중요해.

이 값을 기준으로 **“어느 기상목표를 위해 생성된 알림인지”** 묶을 수 있기 때문에 `잠자기`, 목표시간 변경, 목표 삭제 시 남은 알림만 정확하게 취소할 수 있어.